Innomatics Research Labs

A TECHNICAL DOCUMENTATION REPORT
ON

"RAG-BASED CUSTOMER SUPPORT ASSISTANT"

A Project Report Submitted in Partial Fulfillment of the Requirements for the Agentic AI Internship

Submitted by:

Sushil Panthi
Intern ID: IN226018202

Guided by:
Mr. Mayank Ghai
Trainer | Mentor
Innomatics Research Lab

Innomatics Research Labs

DECLARATION

I, Sushil Panthi, hereby declare that the project entitled "RAG-Based Customer Support Assistant using LangGraph and Human-in-the-Loop (HITL)" is an original work carried out by me as part of the Agentic AI Internship at Innomatics Research Labs.

This technical documentation provides an in-depth explanation of the system architecture, design trade-offs, and implementation reasoning. I confirm that the logic for the LangGraph transitions, ChromaDB local persistence, and HITL escalation was developed to reflect real-world production standards.

Sushil Panthi
Date: April 23, 2026
Location: Nepal

Innomatics Research Labs

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Innomatics Research Labs for providing this challenging and insightful internship opportunity. This project has allowed me to apply theoretical knowledge of Generative AI and Agentic Frameworks into a production-style workflow.

Special thanks to my mentor, Mr. Mayank Ghai, for the technical guidance provided during this project. Also, to the technical team for designing a curriculum that emphasizes system design thinking. Designing the internal logic for an agentic RAG system has provided deep insights into modular software engineering and the practical challenges of Human-AI collaboration.

I also acknowledge the open-source community behind LangChain, LangGraph, and ChromaDB, whose robust APIs form the backbone of this implementation.

Abstract

This Low-Level Design (LLD) document provides a comprehensive technical blueprint for the implementation of the RAG-Based Customer Support Assistant. While the High-Level Design (HLD) established the structural boundaries, this report details the internal mechanics of the system's modular components .

The implementation features a dedicated Document Processing Module that utilizes a recursive chunking strategy (1000 characters with 100-character overlap) to prepare data for the Vector Storage Module. Central to the system is a locally persisted ChromaDB collection, which manages high-dimensional embeddings for efficient semantic retrieval .

A critical technical highlight is the application of LangGraph to manage the agentic state transitions. Unlike linear pipelines, this design defines a specific State Object to track the query, retrieved context, and confidence scores across nodes. The control flow incorporates Conditional Routing Logic, specifically engineered to handle HITL (Human-in-the-Loop) escalation. Escalation is triggered by deterministic criteria, such as low confidence thresholds (< 0.7) or context gaps, and is accompanied by dynamically generated escalation reasons .

The document further outlines the modular prompt architecture, API schemas, and the Streamlit interaction flow, which includes clickable demo questions and real-time processing states to ensure a professional user experience . By focusing on modularity and traceability via LangSmith, this design ensures a robust, production-grade deployment.

Table of Content

Chapter No.	Chapter Name	Page No
1	Introduction to RAG & Project Context	1
2	System Architecture & Design Decisions	2
3	Knowledge Base Handling & ChromaDB	5
4	Workflow Explanation (LangGraph)	6
5	Conditional Logic & Intent Detection	7
6	HITL Implementation & Explainability	8
7	Challenges, Trade-offs & Testing	10
8	Future Enhancements & Conclusion	12

List of Figure

Fig.No	Figure Name	Page No
1	Evolution from Linear RAG to Agentic Workflows: How LangGraph enables state-aware decision making	2
2	Technical Ingestion Lifecycle: The mandatory sequence of PDF parsing, recursive splitting, and vector indexing in ChromaDB	5
3	Deterministic Escalation Logic: Identifying Knowledge Gaps and Low-Confidence scenarios for human intervention	8
4	Engineering Trade-offs: Balancing chunk size and retrieval depth to optimize system groundedness	10

CHAPTER 1: INTRODUCTION TO RAG & PROJECT CONTEXT

a) What is RAG?

Retrieval-Augmented Generation (RAG) is an architectural framework that optimizes the output of a Large Language Model (LLM) by referencing an authoritative, external knowledge base before generating a response. While LLMs are trained on vast datasets, they lack access to proprietary or up-to-date private documents. RAG solves this by "retrieving" relevant snippets from a local database and "augmenting" the user's prompt with this factual context.

b) Why RAG is Needed

In a customer support environment, accuracy is non-negotiable. Standard AI models often suffer from hallucinations, where they confidently state incorrect facts about product policies or technical specifications. RAG is needed to:

- **Ensure Groundedness:** Every answer is anchored in specific excerpts from the provided PDF manuals.
- **Data Privacy:** By using a local RAG setup, sensitive company documents do not need to be used for fine-tuning public models.
- **Cost Efficiency:** RAG provides specialized knowledge without the massive computational expense of retraining an LLM.

CHAPTER 2: SYSTEM ARCHITECTURE & DESIGN DECISIONS

a) Architectural Overview

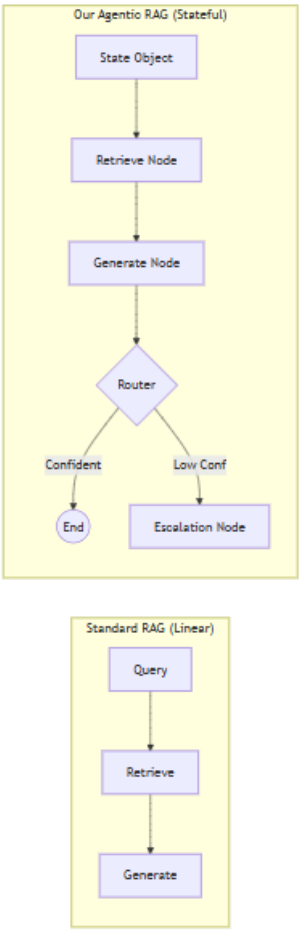

Figure 1 Evolution from Linear RAG to Agentic Workflows: How LangGraph enables state-aware decision making

The system architecture follows a **decoupled, event-driven design** orchestrated by a stateful graph. Unlike traditional RAG applications that treat the process as a single function call, this architecture treats the support interaction as a **managed lifecycle** .

A. The Dual-Pipeline Strategy

The architecture is split into two distinct operational phases to ensure efficiency and scalability :

- **The Static Ingestion Pipeline:** A one-time or periodic process where high-density PDF manuals are transformed into a searchable knowledge base. By using **Recursive Character Splitting**, we preserve the structural hierarchy of technical documents, ensuring headers and sub-points remain contextually linked.

- **The Dynamic Execution Graph:** A real-time loop managed by **LangGraph**. Every user query initializes a new **State Object**, which serves as a container for the data as it moves through the system. This allows for asynchronous processing and the ability to "pause" execution for human intervention.

B. Layered Component Interaction

The architecture is organized into four logical layers to prevent tight coupling:

- **Interface Layer:** Built with **Streamlit**, this layer manages user state and provides immediate visual feedback through node progress spinners and clickable demo questions.
- **Orchestration Layer:** The LangGraph Engine acts as the central controller, using Conditional Routing to decide the path of a query based on intent and confidence scores .
- **Persistence Layer:** ChromaDB functions as the system's "Long-Term Memory," providing local vector indexing and similarity-based retrieval without external dependencies.
- **Inference Layer:** A multi-model strategy utilizing Groq for near-instant primary processing and Google Gemini as a high-reliability fallback for stability.

C. The "Closed-Loop" Safety Mechanism

A defining feature of this architecture is the HITL Safety Loop. By implementing a deterministic routing gate, the system ensures that any query failing the Self-Evaluation (Score < 0.7) is diverted to a human agent . This prevents the "hallucination loop" common in standard RAG systems, making it suitable for high-stakes customer support environments.

b) Design Decisions & Rationale

Why ChromaDB?

A dedicated section for Why ChromaDB is essential to justify our storage strategy:

- **AI-Native & Open Source:** It is built specifically for vector embeddings, offering superior performance over traditional SQL databases for semantic search.
- **Local Persistence:** It allows the assistant to maintain its knowledge base on the local disk, which is better for this use case as it ensures zero-latency network

overhead and total data sovereignty.

Why LangGraph?

We moved beyond linear chains to a graph-based system to support Conditional Logic:

- **Intent Detection:** The system can decide whether a query is a general greeting, a technical question, or a sensitive complaint.
- **Stateful Memory:** The State Object carries the query, retrieved context, and confidence scores throughout the workflow, allowing for iterative processing .

Why Modular Prompting?

Prompt Organization was decoupled from the code to improve maintainability:

- **Ease of Updates:** Support teams can update the "Tone" or "Escalation Guidelines" in a standalone template file without needing to touch the core Python logic.
- **Traceable Logic:** Separating the intent-detection prompt from the response-generation prompt makes it easier to debug exactly where the AI's logic might be failing.

CHAPTER 3: KNOWLEDGE BASE HANDLING

Document Ingestion Pipeline

Figure 2 Technical Ingestion Lifecycle: The mandatory sequence of PDF parsing, recursive splitting, and vector indexing in ChromaDB

The system processes unstructured data through a strictly defined lifecycle:

- **PDF Loading:** The PyPDFLoader extracts text and page metadata.
- **Recursive Chunking:** We chose a 1000-character chunk size with a 100-character overlap.
  - *Trade-off:* Larger chunks provide more context but may dilute the search results; this specific size was selected to balance context quality with retrieval speed.
- **Embedding:** Text is converted into vectors using Google Gemini Embeddings.
- **Storage:** These vectors are indexed into a local ChromaDB collection.

CHAPTER 4: WORKFLOW EXPLANATION (LANGGRAPH)

This chapter provides a deep-dive into the orchestration layer that differentiates this project from a basic RAG script.

a) Role of LangGraph

I utilize LangGraph to transform the retrieval and generation steps into a state-aware agentic workflow. Instead of a linear script, the graph manages a State Object that acts as the system's "short-term memory," carrying variables like the user's query, retrieved context, and confidence scores across different nodes .

b) Node Responsibilities

The workflow is divided into specialized functional units:

- **Retrieval Node:** Interacts with the ChromaDB module to fetch the top 3-5 most relevant document chunks based on semantic similarity.
- **Process/Generation Node:** Takes the retrieved context and utilizes the Grounded RAG Template to synthesize a response. It is also responsible for self-assessing its own output for confidence.
- **HITL Escalation Node:** A safety-critical node that pauses automated execution and prepares a "Handover Packet" for human intervention when criteria are met.

c) State Transitions

Transitions are not hardcoded; they are dynamic. The system evaluates the state after each node execution to determine the next path. For example, if the Retrieval Node fails to find relevant data, the graph skips the Generation Node entirely and routes directly to the Escalation Node, ensuring the LLM never attempts to "guess" without data .

CHAPTER 5: CONDITIONAL LOGIC & INTENT DETECTION

a) Intent Detection

The first layer of intelligence in our graph is the Intent Detection phase. Using a specialized modular prompt, the system analyzes the user's query to categorize it :

- **Standard Support:** Technical questions that can be answered via the PDF.
- **Sensitive/Escalated:** Complaints or legal inquiries requiring immediate human oversight.
- **Out-of-Scope:** Queries unrelated to the documentation.

b) Routing Decisions

The Conditional Routing logic serves as the system's decision-maker. It evaluates the query based on specific Escalation Criteria :

- **Confidence Threshold:** If the LLM generates a response but flags its own confidence score as being below 0.7, the system triggers a routing to HITL.
- **Missing Context:** If the vector search in ChromaDB returns zero relevant chunks, the routing logic identifies a "Knowledge Gap" and escalates.
- **Complexity:** If the query involves multiple steps or conflicting policies, the router identifies it as a "Complex Query" beyond automated resolution.

CHAPTER 6: HITL IMPLEMENTATION & EXPLAINABILITY

a) Human-in-the-Loop Reasoning

The HITL Implementation is designed to balance automated speed with human-level accuracy. By allowing human intervention, we eliminate the liability risks associated with AI hallucinations in sensitive customer support scenarios .

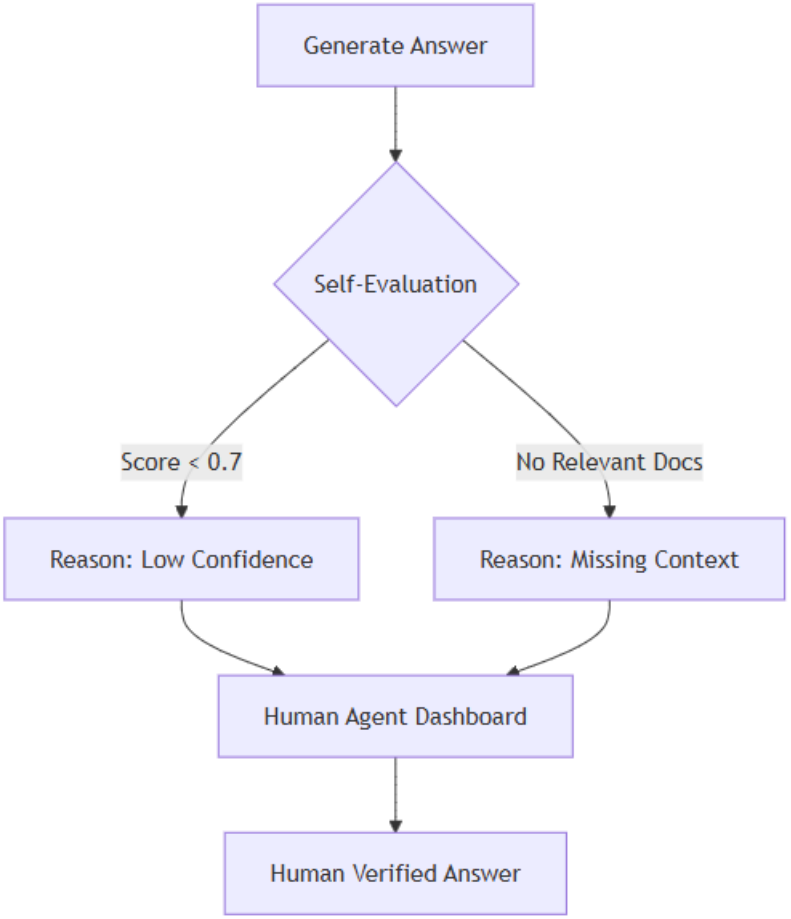

Figure 3 Deterministic Escalation Logic: Identifying Knowledge Gaps and Low-Confidence scenarios for human intervention

b) Explainability

To maintain transparency, the system provides Explainability for every decision:

- **Escalation Reasons:** The assistant returns a human-readable reason (e.g., "Documentation insufficient for safety protocols") to ensure the human agent understands the context of the hand-off.

- **Citations:** For successful RAG responses, the UI displays specific page numbers and sources from the PDF, allowing users to verify the information independently.

CHAPTER 7: CHALLENGES, TRADE-OFFS & TESTING

a) Technical Trade-offs

During the development of this production-style system, several architectural trade-offs were made to balance performance with reliability:

- **Retrieval Accuracy vs. Speed:** Using a larger Top-K (retrieving more chunks) improves context quality but increases the token count and processing time. We settled on K=3 for optimal latency.
- **Chunk Size vs. Context Quality:** We selected a 1000-character chunk size to ensure the LLM receives complete thoughts, accepting the slightly higher storage overhead in ChromaDB.
- **Local vs. Cloud Storage:** Choosing a local vector store (ChromaDB) prioritizes data privacy and zero-cost scaling over the managed features of cloud-hosted alternatives.

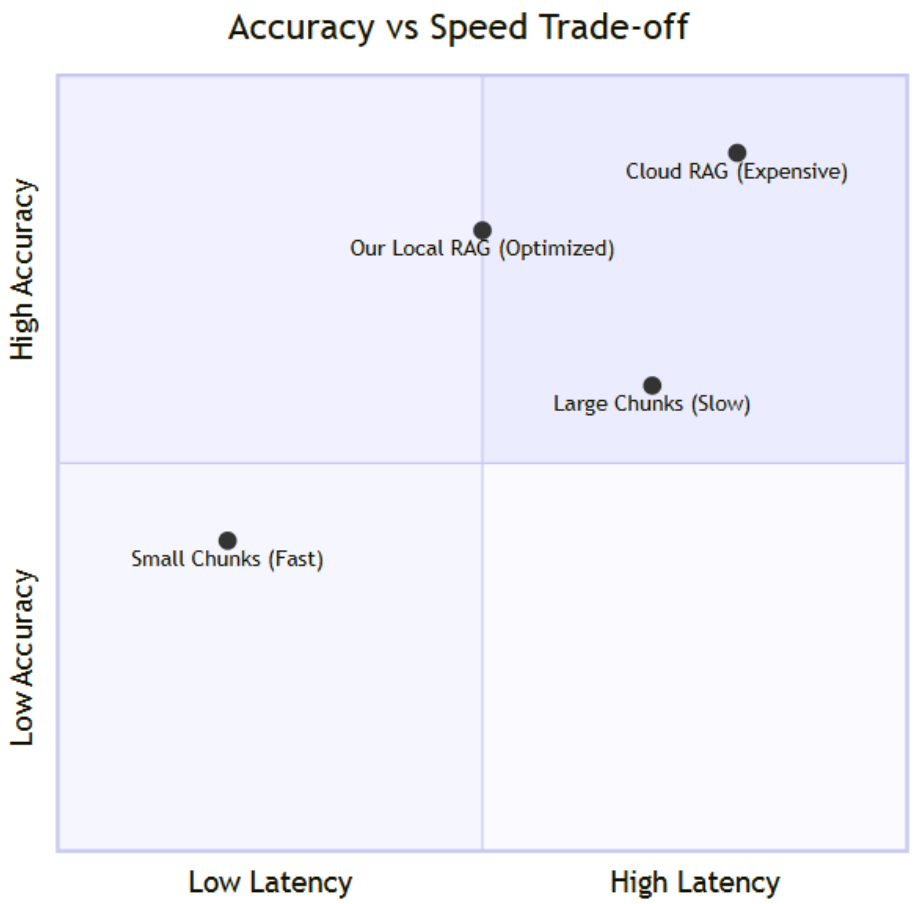

Figure 4 Engineering Trade-offs: Balancing chunk size and retrieval depth to optimize system groundedness

b) Testing Strategy

To ensure the system meets the "Success Criteria," a multi-layered testing approach was implemented:

- **Groundedness Testing:** Using sample queries to verify that the assistant cites the correct page numbers from the PDF knowledge base.
- **Escalation Validation:** Testing the Conditional Routing by asking questions explicitly missing from the manual to confirm the HITL path triggers correctly .
- **UI Stress Testing:** Verifying the Clickable Demo Questions and processing states in Streamlit to ensure a smooth user interaction flow.

Chapter 7: FUTURE ENHANCEMENTS & CONCLUSION

a) Future Scope

The modular architecture of this assistant allows for several future technical upgrades:

- **Multi-Document Support:** Expanding the ingestion pipeline to cross-reference information across an entire library of PDF manuals.
- **Feedback Loops:** Integrating human corrections back into the vector store as "Gold Standard" answers to improve future retrieval accuracy.
- **Memory Integration:** Utilizing **LangGraph's** checkpointer to allow the assistant to remember previous conversation turns for more personalized support.

b) Conclusion

The RAG-Based Customer Support Assistant successfully demonstrates that a production-grade AI system requires more than just a chat interface. By integrating RAG for factual grounding, LangGraph for agentic decision-making, and ChromaDB for private data persistence, we have built a system that is both intelligent and safe. This project proves that the future of customer support lies in the seamless collaboration between automated AI agents and human expertise.

CHAPTER 6: BIBLIOGRAPHY

- **LangChain & LangGraph:** "LangGraph Documentation: Orchestrating Agentic Workflows." *LangChain Inc.*.
- **ChromaDB:** "Chroma: The AI-Native Open-Source Embedding Database." *Chroma Core Team.*.
- **Google Gemini:** "Gemini API Documentation: Embeddings and Generative Models." *Google AI for Developers.*.
- **Groq Cloud:** "Groq LPU Inference Engine: High-Speed LLM Inference." *Groq Inc.*.
- **RAG Best Practices:** "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Meta AI Research.* (2020).